

Relatório final — TaskFlow AI (CloudTask AI SaaS)

Disciplina **Computação em Nuvem** — UNINTER. Projeto executado em **modo local** (Docker), com os recursos de nuvem (EKS, RDS, S3, DynamoDB, CDK) estudados como **conceito + código versionado**. Preencha os campos [...] da identificação.

1. Identificação

- **Aluno(a):** [SEU NOME COMPLETO]
- **RU / matrícula:** [SEU RU]
- **Disciplina:** Computação em Nuvem — UNINTER
- **Repositório:** <https://github.com/JlucasFaria/computacao-em-nuvem-trabalho-final> (branch joaoLucas-trabalho-final)
- **Data:** [dd/mm/aaaa]

2. Resumo do projeto

O **TaskFlow AI** é um mini-SaaS de **gerenciamento de tarefas** construído ao longo da disciplina. Oferece uma **API REST** (FastAPI, com Swagger interativo) para CRUD de tarefas, **autenticação por token JWT**, **upload de arquivos** e registro de **eventos/logs** de auditoria. A stack é **Python + FastAPI + PostgreSQL**, empacotada em **Docker**, com caminhos de nuvem (Kubernetes/EKS, ECR, S3, DynamoDB) e **infraestrutura como código (AWS CDK)**. Todo recurso de nuvem tem um **fallback local**, permitindo rodar o projeto inteiro sem AWS.

3. O que foi implementado (por semana)

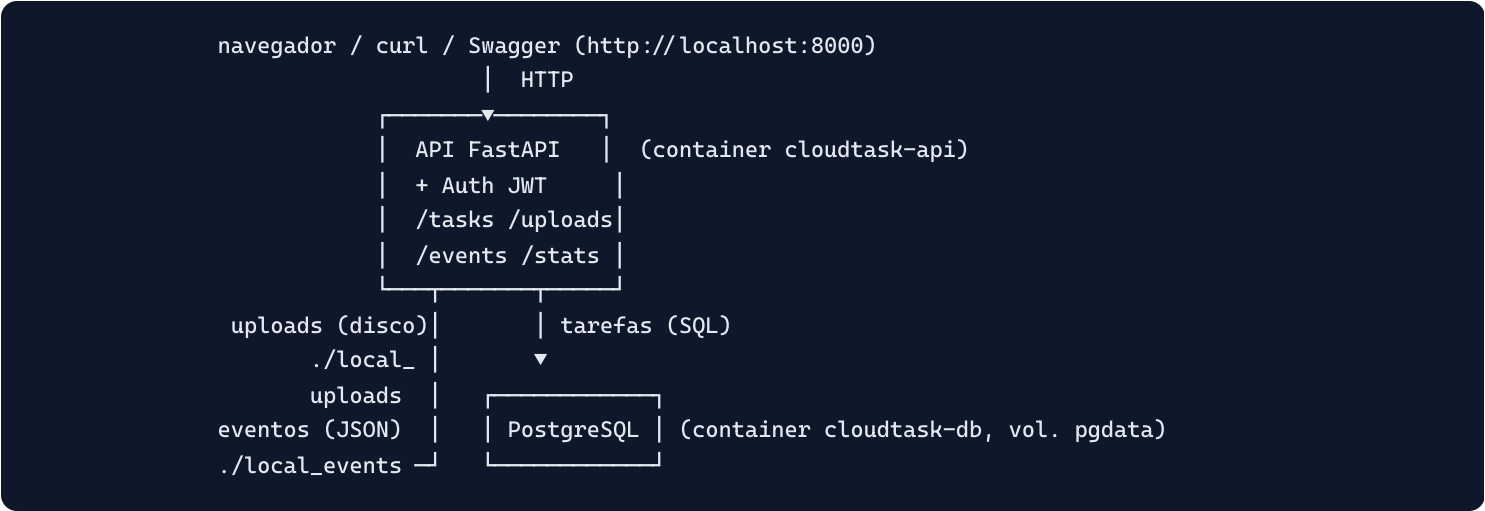
Executado e validado em **modo local** (Docker Compose: API + PostgreSQL 16). Evidências em docs/entrega-final/evidencias/.

Semana	Entreguei	Evidência (comando / endpoint)
1 — FastAPI + Docker	API FastAPI com <code>/</code> , <code>/health</code> , Swagger; imagem Docker + Compose	<code>GET /health</code> → <code>{"status":"ok"}</code> ; <code>docker compose up</code>
2 — PostgreSQL + config	CRUD de tarefas no PostgreSQL; config via <code>.env</code> /pydantic-settings; readiness	<code>GET /health/ready</code> → <code>{"status":"ready", "db":"ok"}</code> ; <code>POST/GET/PUT/DELETE /tasks</code>
3 — S3 + Kind	Upload com backend selecionável (modo local validado); manifests Kubernetes	<code>POST /uploads</code> → <code>{"storage_mode":"local"}</code> ; <code>infra/k8s/</code>
4 — ECR + EKS	Script de build/push para ECR e manifests EKS (caminho AWS, como conceito)	<code>scripts/semana-04-ecr/</code> ; <code>infra/k8s/aws/</code>
5 — HPA + DynamoDB	Eventos automáticos em create/update/delete (modo local JSON validado); HPA + teste de carga	<code>GET /events</code> → <code>task.created</code> ; <code>infra/k8s/hpa.yaml</code>
6 — CDK + entrega	7 stacks CDK (S3, ECR, VPC, DynamoDB, etc.), auth JWT, docs finais	<code>POST /auth/login</code> → JWT; <code>infra/cdk/</code>
Extra (meu toque pessoal)	Rebranding → TaskFlow AI + endpoint <code>GET /tasks/stats</code> (agregação) + 3 testes	<code>GET /tasks/stats</code> → total + contagem por status/prioridade

Testes automatizados: `docker compose exec -T api pytest` → **73 passed** (70 da base + 3 do endpoint novo).

4. Arquitetura

O que EU subi (modo local):



Arquitetura-alvo em produção (conceito, coberto no código/CDK): a mesma API rodaria em **EKS** (imagem no **ECR**, HPA 2→5), atrás de **ALB + ACM** (HTTPS na borda), com **RDS** (tarefas), **S3** (uploads) e **DynamoDB** (eventos) — tudo descrito como código no **AWS CDK** (`infra/cdk/`). Ver `final-architecture.md` .

5. Como executar (reprodutível)

```
# pré-requisito: Docker Desktop rodando
cp .env.example .env
docker compose up --build -d          # sobe API + PostgreSQL

# healthcheck
curl http://localhost:8000/health      # {"status":"ok"}
curl http://localhost:8000/health/ready  # {"status":"ready","db":"ok"}

# autenticar (rotas de dados exigem token JWT)
TOKEN=$(curl -s -X POST http://localhost:8000/auth/login \
  -H "Content-Type: application/json" \
  -d '{"username":"admin","password":"admin#123"}' | \
  python -c "import sys,json;print(json.load(sys.stdin)['access_token'])")

# usar a API
curl -X POST http://localhost:8000/tasks -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"title":"Estudar Cloud","priority":"high"}'
curl http://localhost:8000/tasks/stats -H "Authorization: Bearer $TOKEN"

# testes
docker compose exec -T api pytest      # 73 passed

# Swagger interativo: http://localhost:8000/docs
```

6. Decisões e trade-offs

- **PostgreSQL como container, não RDS:** sem custo e mais simples para a demo. Trade-off: perco backup gerenciado, HA e patch automático que o RDS traria.
- **Modo local para S3 e eventos (fallback):** permite rodar o projeto inteiro **sem AWS/credenciais**. Trocar para S3/DynamoDB é só mudar o `.env` .
- **Autenticação JWT no próprio backend:** simplificação didática. Em produção, um provedor de identidade dedicado (Cognito/OAuth) centralizaria o login.
- **HTTPS na borda (conceito):** TLS terminaria no ALB/Edge, nunca no app — evita loop de redirect nas health probes. Localmente rodo em HTTP puro.
- **Endpoint `/tasks/stats` agrega no banco (COUNT/GROUP BY):** mais eficiente que trazer todas as linhas para a memória da API. Declarado **antes** de `/tasks/{id}` para o FastAPI não tratar `"stats"`

como um id.

7. Custos

- **Recursos que cobraram: nenhum.** Execução 100% local (Docker na própria máquina).
- **Estimativa do período: US\$ 0,00.**
- **Confirmação de limpeza:** não se aplica — nada foi provisionado na AWS, portanto não há recurso cobrável para destruir. Sweep detalhado no `deployment-checklist-preenchido.md`.

8. LGPD e segurança

- A aplicação coleta **pouquíssimo dado pessoal** (tarefas são texto livre; uma única conta administrativa). Dados em PostgreSQL/disco local.
- **Segredos protegidos:** `.env` no `.gitignore` e **não** versionado (verificado); **nenhuma chave AWS real** no repositório. A senha `admin#123` é credencial de **demo** documentada.
- **Direitos do titular:** acesso via `GET /tasks`, exclusão via `DELETE /tasks`.
- **Em produção real ficaria pendente:** TLS na borda (ACM), criptografia em repouso (S3/RDS), segredos no Secrets Manager, roles IAM de menor privilégio.
- Detalhes no `lgpd-checklist-preenchido.md`.

9. Dificuldades e aprendizados

- **Autenticação inesperada:** ao rodar, descobri que as rotas de dados agora exigem **token JWT** (Semana 6). Apreendi a fazer login e usar o header `Authorization: Bearer`. (*Lição: sempre validar o fluxo real antes de assumir.*)
- **Ordem de rotas no FastAPI:** ao criar `/tasks/stats`, entendi que rotas específicas precisam vir **antes** das paramétricas (`/tasks/{id}`), senão o framework confunde `"stats"` com um id → erro 422.
- **Agregação no banco:** usar `COUNT` + `GROUP BY` em vez de contar em Python.
- **Fallback local:** compreendi na prática o valor de projetar com fallback — consegui rodar e demonstrar tudo **sem gastar 1 centavo** na AWS.
- **Cuidado com segredos:** verifiquei de fato (não "no chute") que nada sensível foi para o git antes de publicar o repositório.

10. Anexos

- ☒ `lgpd-checklist-preenchido.md` preenchido

- ☒ `deployment-checklist-preenchido.md` (custos/limpeza) preenchido
- ☒ Evidências: `evidencias/01-smoke-test-local.md`, `evidencias/02-toque-pessoal.md`
- ☐ Prints do Swagger (`http://localhost:8000/docs`) — **anexar na hora de gerar o PDF**

Anexo A — Evidências de execução (Passo 1)

Evidências — Passo 1: execução local (smoke test)

Ambiente: Windows 11 + Docker Desktop. Modo **local** (fallback sem AWS): `STORAGE_MODE=local`, eventos em JSON. Data: 02/07/2026.

Como foi subido

```
docker compose up --build -d      # sobe API (FastAPI) + PostgreSQL 16
docker compose ps                 # cloudtask-api + cloudtask-db (healthy)
```

1. Metadados, liveness e readiness

```
GET /          → {"name":"CloudTask AI SaaS","version":"0.6.0","docs":"/docs"}
GET /health    → {"status":"ok"}
GET /health/ready → {"status":"ready","db":"ok"}      # readiness checkou o Postgres
```

2. Autenticação (JWT — Semana 6)

```
POST /auth/login {"username":"admin","password":"admin#123"}
→ {"access_token":"eyJhbGciOiJIUzI1NiI...","token_type":"bearer","expires_in":28800}
```

As rotas de dados exigem `Authorization: Bearer <token>` (retornam 401 sem token).

3. CRUD de tarefas (Semana 2)

```
POST /tasks {"title":"Minha primeira tarefa","description":"...","priority":"high"}
→ 201 {"id":1,"status":"pending","priority":"high", ...}
GET /tasks
→ [ {"id":1,"title":"Minha primeira tarefa", ...} ]
```

4. Upload de arquivo — modo local (Semana 3)

```
POST /uploads (multipart, file=@README.md)
→ {"filename":"2d923ad12998f5a9-ddd81d19.md","url":"/uploads/ ... ","storage_mode":"local"}
```

5. Eventos automáticos (Semana 5 / Aula 10)

```
GET /events
→ [ {"event_type":"task.created","task_id":1,
      "message":"Tarefa 1 criada: 'Minha primeira tarefa'.", ...} ]
```

O evento `task.created` foi gerado **automaticamente** ao criar a tarefa — integra CRUD (Aula 3) + eventos (Aula 10).

6. Testes automatizados

```
docker compose exec -T api pytest
# ..... [100%]
# 70 passed in 1.89s
```

Conclusão

Aplicação sobe e funciona **ponta a ponta em modo local**, sem depender da AWS, conforme o princípio de *fallback local* definido pelo professor em `docs/TAREFAS.md`. Todos os endpoints das Aulas 1–12 respondem; 70 testes automatizados passam.

Anexo B — Evidências do toque pessoal (Passo 2)

Evidências — Passo 2: toque pessoal

Customizações feitas por mim sobre a base, para o projeto não ser cópia idêntica e mostrar domínio do código. Data: 02/07/2026.

1. Rebranding: CloudTask AI SaaS → TaskFlow AI

Alterado o nome visível do serviço no título do Swagger, no endpoint raiz e na configuração.

Arquivos: `app/main.py` (título FastAPI, endpoint `/`, descrição), `app/core/config.py` (`app_name`), `tests/test_health.py` (asserção do nome).

```
GET / → {"name":"TaskFlow AI","version":"0.6.0","docs":"/docs"}
```

O título **TaskFlow AI** aparece no topo do Swagger UI (`/docs`).

2. Endpoint novo: GET /tasks/stats (não existia na base)

Retorna um **resumo agregado** das tarefas — total geral + contagem por `status` e por `priority`. Feito com `COUNT` + `GROUP BY` no PostgreSQL (agregação no banco, não em Python).

Arquivos: `app/api/routes_tasks.py` (endpoint + query), `app/db/schemas.py` (schema `TaskStats`), `tests/test_tasks_crud.py` (3 testes).

Detalhe técnico: o endpoint é declarado **antes** de `/tasks/{task_id}` para o FastAPI não interpretar `"stats"` como um id (evita erro 422).

```
GET /tasks/stats
→ {"total":3,
    "by_status":{"pending":1,"in_progress":1,"done":1},
    "by_priority":{"low":1,"medium":0,"high":2}}
```


3. Testes

Adicionei 3 testes automatizados para o novo endpoint (`TestStats`): vazio, contagem por status/prioridade, e a não-colisão com a rota `/task_id` .

```
docker compose exec -T api pytest
# 73 passed in 2.01s      (70 da base + 3 novos)
```

Toda a suíte continua verde — a customização não quebrou nada da base.

Checklist LGPD + segurança — TaskFlow AI (PREENCHIDO)

Preenchido por **João Lucas** em 02/07/2026. Projeto executado em **modo local** (Docker, sem AWS). Itens específicos de nuvem são marcados como "*não se aplica — modo local*" com a explicação de **como seria em produção real**, mostrando entendimento do conceito. Baseado em `lgpd-checklist.md` (template do professor).

1. Dados pessoais — mapeamento

- ☒ **Quais** dados pessoais a aplicação coleta: **praticamente nenhum**. As tarefas são texto livre (`title / description`); só haveria dado pessoal se o usuário digitasse um no texto. A autenticação usa **uma única conta administrativa** (`admin`), sem cadastro de titulares.
- ☒ **Onde** cada dado é armazenado: tarefas no **PostgreSQL**; uploads no **disco local** (`./local_uploads` , modo local); eventos em **JSON local** (`./local_events/events.json`). Em produção: RDS + S3 + DynamoDB.
- ☒ **Por quanto tempo / como apagar**: não há retenção automática. O titular apaga via `DELETE /tasks/{id}` . Sem política formal de expiração (seria definida em produção real).

2. Bases legais e finalidade (LGPD art. 6–11)

- ☒ **Finalidade específica**: gerenciar tarefas (estudo/demonstração da disciplina). Documentada aqui e no README.
- ☒ **Base legal**: projeto **didático** — a finalidade documentada cumpre o exercício. Em uso real, seria execução de contrato / legítimo interesse.

3. Segurança técnica (LGPD art. 46)

- [~] **Em trânsito**: em modo local roda em **HTTP** (`localhost` , dev). *Não se aplica TLS localmente*. Em produção real, o TLS termina na **borda** (Edge Caddy com cert ACME, ou ALB + ACM) — a API nunca administra certificado. Conceito coberto em `docs/conceitos/https-tls.md` .
- [~] **Em repouso**: disco local sem criptografia (dev). Em produção: S3 (`S3_MANAGED`), RDS (encryption at rest), DynamoDB (padrão) — todos com criptografia ativa.
- ☒ **Segredos fora do código/git: verificado** — `.env` está no `.gitignore` e **não** é rastreado (`git ls-files` não lista `.env`); nenhuma chave AWS real (`AKIA...`) no repositório. A senha de demo

`admin#123` é uma **simplificação didática** documentada (default em `config.py`); em produção viria de Secrets Manager / SSM e nunca seria fixa.

- [~] **Bucket S3 privado (Block Public Access)**: não se aplica (modo local). As stacks CDK (`infra/cdk/`) criam o bucket **privado e criptografado** — conceito coberto.
- [~] **Credenciais temporárias (roles)**: não se aplica localmente. Em AWS, o EC2/EKS usa **roles** (sem chave fixa) — ver `infra/cdk` e docs.
- ☒ **Menor privilégio**: a aplicação acessa só o banco/armazenamento que precisa; rotas de dados exigem token JWT.

4. Direitos do titular (LGPD art. 18)

- ☒ **Acessar** os dados: `GET /tasks` (listar) e `GET /tasks/{id}` (consultar).
- ☒ **Excluir**: `DELETE /tasks/{id}` remove a tarefa. (Uploads podem ser removidos do armazenamento correspondente.)
- ☒ **Logs sem dado sensível**: os eventos (`task.created/updated/deleted`) guardam só id, tipo e uma mensagem curta — **não** copiam conteúdo sensível.

5. Operação e incidentes

- [~] **Backups**: em modo local, os dados vivem no volume `pgdata` do Docker. Em produção: RDS com snapshot automático + S3 versionado (conceito coberto).
- ☒ **Reação a vazamento**: rotacionar `SECRET_KEY` (invalida os JWT), trocar a senha admin, e — em produção — rotacionar segredos no Secrets Manager.
- [~] **Cost/uso monitorado (Budgets)**: não se aplica (sem gasto AWS). Conceito em `docs/conceitos/cost-explorer.md`.

6. Higiene de projeto

- ☒ **Nenhuma conta AWS real ou segredo commitado**: verificado (seção 3).
- ☒ **Recursos de teste destruídos**: não se aplica — nada foi provisionado na AWS (modo local). Sem recurso órfão cobrável.
- ☒ **README/docs não expõem credenciais internas**: só a conta de **demo** documentada (`admin / admin#123`), intencional para a avaliação.

Legenda: `[x]` feito/verificado · `[~]` não se aplica ao modo local (explicado como seria em produção).

Checklist de deploy + custos — TaskFlow AI (PREENCHIDO)

Preenchido por **João Lucas** em 02/07/2026. Execução em **modo local** (Docker Compose: API FastAPI + PostgreSQL 16). **Nada foi provisionado na AWS**, então os blocos de nuvem (EKS/RDS/ELB/CDK) são marcados "*não se aplica*" com a explicação de como seriam em produção. Baseado em `deployment-checklist.md`.

Antes do deploy

- [~] Credenciais AWS válidas (`aws sts get-caller-identity`): **não se aplica** — execução local, sem AWS.
- [~] Região correta (`us-east-1`): não se aplica (local).
- [~] Imagem no ECR: não se aplica. A imagem foi construída **localmente** (`docker compose build` , `cloudtask-api:dev`). Em produção iria para o ECR via `scripts/semana-04-ecr/build-push-ecr.sh`.
- ☒ `.env` revisado: criado a partir de `.env.example`, sem placeholders quebrados.
- ☒ Banco definido: **PostgreSQL 16 como container** (serviço `db` do Compose), ciente do trade-off vs. RDS (RDS traria backup gerenciado, HA, patch automático; o container é mais simples e sem custo, ideal para a demo).

Durante (subir)

- ☒ Aplicação sobe sem erro: `docker compose up --build -d` → `cloudtask-api` + `cloudtask-db` (este último `healthy`).
- [~] Cluster EKS `Ready` / `metrics-server` / `kubectl apply` : **não se aplica** (local). Manifests existem em `infra/k8s/aws/` para o caminho AWS.
- ☒ Serviço acessível: `curl http://localhost:8000/health` → `200` ; `/health/ready` → `{"status": "ready", "db": "ok"}` (readiness checkou o Postgres).

Verificação funcional (demonstrar)

- ☒ **Autenticação**: `POST /auth/login` (admin/admin#123) devolve JWT; rotas de dados exigem `Authorization: Bearer <token>`.
- ☒ **CRUD**: criar/listar/atualizar/excluir tarefa (Swagger ou curl) — OK.

- ☒ **Upload:** `POST /uploads` em **modo local** grava em `./local_uploads` e devolve nome + URL; download OK.
- ☒ **Evento:** criar tarefa gera `task.created` (`GET /events`) — OK.
- ☒ **Extra (meu):** `GET /tasks/stats` devolve total + contagem por status e prioridade — OK.
- [~] **HPA** (gerar carga e ver réplicas): não se aplica (local, sem cluster).

🔥 Depois (destruir — OBRIGATÓRIO na nuvem)

- [~] Todos os itens (`kubectl delete` , `eksctl delete cluster` , apagar RDS/ DynamoDB/S3, `cdk destroy`): **não se aplica** — nada foi criado na AWS, portanto **não há recurso cobrável para destruir**.
- ☒ Ambiente local encerrado quando necessário: `docker compose down` (usar `down -v` zera o volume `pgdata`).

Sweep final (tudo vazio = zero cobrança)

- [~] Comandos de varredura (EKS/EC2/ELB/NAT/EIP/RDS): **não se aplica** (local). Como nada foi provisionado, o resultado de todos seria vazio por definição.
- ☒ **Custo do período: US\$ 0,00** — execução 100% local, sem uso de AWS.

Resumo de custos

Recurso	Cobrou?	Observação
Tudo (execução local)	Não	Docker na própria máquina; US\$ 0,00
EKS / EC2 / RDS / ELB / S3 / DynamoDB	Não	Não provisionados — caminho AWS ficou como conceito

Legenda: [x] feito · [~] não se aplica ao modo local (explicado).